Artificial Neural Networks and Deep Learning

Lab Session

Sébastien Harispe
sebastien.harispe@mines-ales.fr

IMT Mines Alès

September 21, 2026

v1

Questions: MLPs and Neural Network Training

Questions: MLPs and Neural Network Training

Topic 1 — Network Architecture and Forward Propagation

Question 1 – Network Architecture [Level 1]

Consider the following MLP with $\mathbf{x} \in \mathbb{R}^{20}$:

$$\mathbf{x} \to \text{Linear}(20, 64) \to \text{ReLU} \to \text{Linear}(64, 32) \to \text{ReLU} \to \text{Linear}(32, 3)$$

Questions

1. What is the dimension of the network output?
2. How many trainable parameters does the network contain?
3. What is the role of the two ReLU layers?

Answer 1 – Network Architecture

The output dimension is

$$\boxed{3}.$$

The number of parameters is

$$(20 \times 64 + 64) + (64 \times 32 + 32) + (32 \times 3 + 3)$$

$$= 1344 + 2080 + 99 = \boxed{3523}.$$

ReLU introduces nonlinearity. Without nonlinear activations, a composition of linear/affine layers would remain affine.

Question 2 – Parameter Counting [Level 1]

Consider a fully connected layer

$$\mathbb{R}^{50} \to \mathbb{R}^{12}.$$

Question

How many trainable parameters does this layer contain if it has one bias for every output neuron?

Answer 2 – Parameter Counting

There are

$$50 \times 12 = 600$$

weights and

$$12$$

biases.
Therefore:

$$\boxed{612}$$

trainable parameters.

Question 3 – Why Nonlinearity? [Level 1]

Consider a network composed exclusively of affine layers:

$$\mathbf{x} \to W_1\mathbf{x} + \mathbf{b}_1 \to W_2(\cdot) + \mathbf{b}_2 \to W_3(\cdot) + \mathbf{b}_3.$$

Question

Is this network more expressive than a single affine layer?
Explain mathematically.

Answer 3 – Why Nonlinearity?

No.
The composition can be written

$$f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$$

for some W and $\mathbf{b}$.
For example,

$$W = W_3 W_2 W_1.$$

Thus depth alone does not make the function nonlinear.

Key point

Nonlinear activation functions are essential for obtaining nonlinear function classes.

Question 4 – Forward Propagation [Level 1]

Consider one neuron:

$$z = w_1 x_1 + w_2 x_2 + b$$

with

$$x_1 = 2, \qquad x_2 = -1,$$

$$w_1 = 0.5, \qquad w_2 = 1, \qquad b = -0.5.$$

The activation function is

$$a = \max(0, z).$$

Question

Compute z and a.

Answer 4 – Forward Propagation

$$z = (0.5)(2) + (1)(-1) - 0.5 = -0.5.$$

Therefore,

$$a = \max(0, -0.5) = \boxed{0}.$$

The neuron is inactive for this input.

Question 5 – Activation Functions [Level 2]

Consider a neuron using ReLU:

$$a = \max(0, z).$$

Questions

1. What is the derivative of ReLU for $z < 0$?
2. What happens to the gradient through this neuron when $z < 0$?
3. What problem can arise if a neuron remains in this region for most inputs during training?

Answer 5 – Activation Functions

Note that

$$\boxed{\mathrm{ReLU}'(x) = \begin{cases} 0 & \text{if } x \leq 0, \\ 1 & \text{if } x > 0. \end{cases}}$$

$\mathrm{ReLU}'(0) = 0$ is a convention.
For $z < 0$,

$$\frac{da}{dz} = 0.$$

Therefore the gradient propagated through the ReLU is zero.
If a neuron remains inactive for most inputs, its parameters may receive little or no gradient. This is related to the **dying ReLU** phenomenon.

Questions: MLPs and Neural Network Training

Topic 2 — Prediction Problems and Output Representations

Question 6 – Regression or Classification? [Level 1]

A model must predict the type of defect observed on a manufactured part:

$$y \in \{\text{crack}, \text{scratch}, \text{deformation}, \text{none}\}.$$

An engineer proposes

$$\text{Linear}(64, 1).$$

Question

Is this an appropriate output representation?
If not, propose a standard alternative.

Answer 6 – Regression or Classification?

This is a multiclass classification problem with four mutually exclusive classes.
A standard formulation is

$$\boxed{\text{Linear}(64, 4)}$$

producing four logits.
The predicted class is

$$\hat{y} = \arg\max_k z_k.$$

A typical loss is

`nn.CrossEntropyLoss()`.

Question 7 – Choosing the Output Layer [Level 2]

For each problem, propose an appropriate output formulation and typical loss.

1. Predict the temperature of a machine.
2. Predict whether a machine will fail.
3. Predict one of five mutually exclusive fault types.
4. Predict the presence of four independent faults.

Answer 7 – Choosing the Output Layer

Problem	**Output**	**Typical loss**
Temperature	1 scalar	MSE / MAE
Binary failure	1 logit	BCEWithLogitsLoss
5-class classification	5 logits	CrossEntropyLoss
4 independent faults	4 logits	BCEWithLogitsLoss

The last two cases illustrate the distinction

$$\boxed{\text{multiclass} \neq \text{multilabel}}.$$

Question 8 – Probabilities or Logits? [Level 1]

A binary classifier produces values such as

$$-2.4, \quad 0.7, \quad 3.1.$$

Questions

1. Can these values directly be probabilities?
2. What are they likely to represent?
3. How can they be converted into probabilities?

Answer 8 – Probabilities or Logits?

They cannot directly be probabilities because probabilities belong to $[0, 1]$.
They are naturally interpreted as **logits**.
For binary classification,

$$p(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

Thus:

$$\boxed{\text{logit} \rightarrow \text{sigmoid} \rightarrow \text{probability}}.$$

During training, the sigmoid is normally incorporated into `BCEWithLogitsLoss`.

Question 9 – Classification Output [Level 1]

An MLP performs classification into four classes.
The final layer is

$$\text{Linear}(64, 4).$$

Questions

1. What does the network output represent?
2. Should the four values necessarily sum to 1?
3. How are probabilities obtained?
4. What happens when using `nn.CrossEntropyLoss()`?

Answer 9 – Classification Output

The output is

$$\mathbf{z} \in \mathbb{R}^4,$$

containing four logits.
The logits do not need to sum to one.
Probabilities are obtained through softmax:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

With `nn.CrossEntropyLoss()`,
the logits should normally be passed directly to the loss.

Common mistake

Do not normally apply an explicit Softmax before `CrossEntropyLoss`.

Question 10 – Binary Classification Modelling [Level 2]

An engineer wants to predict whether a machine will fail:

$$y \in \{0, 1\}.$$

They use

```
model = nn.Sequential(
            nn.Linear(20, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

criterion = nn.MSELoss()
```

Questions

1. Is this mathematically possible?
2. Is it the standard formulation for binary classification?

Answer 10 – Binary Classification Modelling

MSE can be used with a scalar output and targets encoded as 0/1, but it is not the standard likelihood-based formulation for a Bernoulli classification problem.
A standard approach is

$$z = f_\theta(x) \in \mathbb{R}$$

with

$$\boxed{\texttt{BCEWithLogitsLoss}}.$$

The model outputs the logit directly:

$$\text{logit} \rightarrow \text{BCEWithLogitsLoss}.$$

The sigmoid can be applied afterwards when a probability is needed.

Question 11 – Harder: Why BCE Instead of MSE? [Level 3]

For binary classification, suppose the target is $y \in \{0, 1\}$ and the model predicts

$$p = \sigma(z).$$

Compare the following two losses:

$$L_{\text{MSE}} = (y - p)^2$$

and

$$L_{\text{BCE}} = -y \log p - (1 - y) \log(1 - p).$$

Question

Why is BCE particularly natural for binary classification?
What probabilistic assumption does it correspond to?

Answer 11 – Why BCE Instead of MSE?

BCE is the negative log-likelihood of a Bernoulli model:

$$y \sim \text{Bernoulli}(p).$$

Therefore minimizing BCE corresponds to maximum likelihood estimation of the conditional probability

$$p(y = 1 \mid x).$$

MSE instead corresponds naturally to a conditional mean regression objective under a different modelling interpretation.
MSE is therefore not impossible for binary targets, but BCE is the standard probabilistic formulation when the target is Bernoulli.

Questions: MLPs and Neural Network Training

Topic 3 — Backpropagation and Optimization

Question 12 – Gradient Descent [Level 1]

The loss with respect to a parameter θ is $L(\theta)$.
At the current value,

$$\frac{\partial L}{\partial \theta} = 4, \qquad \eta = 0.1, \qquad \theta = 2.$$

Question

Using gradient descent, compute the new value of θ.

Answer 12 – Gradient Descent

$$\theta_{\text{new}} = \theta - \eta \frac{\partial L}{\partial \theta}.$$

Therefore

$$\theta_{\text{new}} = 2 - (0.1)(4) = \boxed{1.6}.$$

Question 13 – Backpropagation [Level 1]

Consider a neural network with parameters θ and loss

$$L(\theta).$$

Questions

1. What is the purpose of backpropagation?
2. What mathematical principle does it rely on?
3. What quantity does it provide to the optimizer?

Answer 13 – Backpropagation

Backpropagation efficiently computes

$$\boxed{\nabla_\theta L}.$$

It relies on the chain rule.
For example,

$$z = f(x), \qquad a = g(z), \qquad L = \ell(a),$$

gives

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}.$$

Backpropagation computes the derivatives required to obtain the gradients of the parameters.

Question 14 – Training Loop [Level 1]

Consider:

```
optimizer.zero_grad()

logits = model(x)

loss = criterion(logits, y)

loss.backward()

optimizer.step()
```

Question

Explain the role of each line.

Answer 14 – Training Loop

1. `zero_grad()` clears accumulated gradients.
2. `model(x)` performs the forward pass.
3. `criterion(...)` computes the loss.
4. `loss.backward()` computes gradients.
5. `optimizer.step()` updates the parameters.

Conceptually:

$$\boxed{\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backward} \rightarrow \text{Update}}$$

Question 15 – Conceptual Synthesis [Level 2]

An engineer says:

> *"The optimizer performs backpropagation to calculate the gradients, and backpropagation uses the gradients to update the weights."*

Question

Identify the conceptual error.

Answer 15 – Conceptual Synthesis

The roles are reversed.

$$\boxed{\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Gradients} \rightarrow \text{Optimizer} \rightarrow \text{Update}}$$

Backpropagation computes gradients.
The optimizer uses the gradients to update the parameters.

Question 16 – Harder: Chain Rule [Level 3]

Consider

$$z = wx + b,$$

$$a = \mathrm{ReLU}(z),$$

and

$$L = (a - y)^2.$$

Questions

Derive

$$\frac{\partial L}{\partial w}.$$

What happens to this gradient when $z < 0$?

Answer 16 – Chain Rule

Using the chain rule,

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w}.$$

We have

$$\frac{\partial L}{\partial a} = 2(a - y),$$

$$\frac{\partial a}{\partial z} = \mathbf{1}_{z>0},$$

and

$$\frac{\partial z}{\partial w} = x.$$

Therefore,

$$\boxed{\frac{\partial L}{\partial w} = 2(a - y)\mathbf{1}_{z>0}x}.$$

When $z < 0$, the ReLU derivative is zero, so $\boxed{\frac{\partial L}{\partial w} = 0}$.

Questions: MLPs and Neural Network Training

Topic 4 — Batches, Epochs and Optimization Dynamics

Question 17 – Batch, Iteration and Epoch [Level 1]

A training set contains

$$N = 10\,000$$

examples and the batch size is

$$B = 100.$$

Questions

1. How many batches are processed during one epoch?
2. Assuming one optimizer update per batch, how many updates occur during one epoch?
3. What is an iteration?
4. How many examples are processed after 5 epochs?

Answer 17 – Batch, Iteration and Epoch

$$\frac{10\,000}{100} = \boxed{100}$$

batches per epoch.
Thus there are

$$\boxed{100}$$

optimizer updates per epoch under the stated training loop.
After 5 epochs,

$$5 \times 10\,000 = \boxed{50\,000}$$

example presentations occur.
An iteration usually corresponds here to one batch-processing step and one optimizer update.

Question 18 – Batch Size [Level 2]

Compare

$$B = 16$$

and

$$B = 512.$$

Question

Describe at least two important consequences of using the larger batch size.

Answer 18 – Batch Size

For a fixed dataset, a larger batch gives:

- fewer parameter updates per epoch;
- a less noisy estimate of the batch gradient;
- greater memory requirements.

Batch size therefore affects optimization dynamics, not only memory usage.

Question 19 – Epoch vs Iteration [Level 1]

An engineer says:

> *"The model has been trained for 100 iterations, so it has seen the complete training dataset 100 times."*

Question

Is this necessarily correct?

Answer 19 – Epoch vs Iteration

No.
An iteration generally corresponds to one batch, whereas an epoch corresponds approximately to one complete pass through the dataset.
The relationship is approximately

$$\text{iterations per epoch} \approx \frac{N}{B},$$

subject to details such as incomplete batches and sampling.

Question 20 – Harder: Gradient Accumulation [Level 3]

Consider:

```
optimizer.zero_grad()

for x, y in dataloader:
    loss = criterion(model(x), y)
    loss.backward()

optimizer.step()
```

Question

How does this differ from calling `optimizer.step()` after every batch? What happens to the gradients?

Answer 20 – Gradient Accumulation

Calling `loss.backward()` accumulates gradients in the parameter's `.grad` fields.

Therefore the optimizer receives a gradient corresponding to the sum of the gradients from all processed batches, unless the loss has been appropriately normalized.

The code therefore performs a form of **gradient accumulation**.

This is useful when the desired effective batch size is larger than the available GPU memory.

Questions: MLPs and Neural Network Training

Topic 5 — Learning Rate and Training Diagnosis

Question 21 – Learning Rate [Level 1]

Two experiments produce:

	Learning rate	Training behavior
A	10^{-5}	Very slow decrease
B	10^{0}	Highly unstable

Question

What do these observations suggest? What would you try next?

Answer 21 – Learning Rate

Experiment A suggests that the learning rate may be too small.
Experiment B suggests that it may be too large, producing overshooting or instability.
A reasonable strategy is to test intermediate values, preferably on a logarithmic scale.
Validation performance must also be considered.

Question 22 – Diagnosing Training [Level 2]

Suppose:

	Training	Validation
Loss	$\downarrow\downarrow$	$\uparrow$
Accuracy	$\uparrow\uparrow$	$\downarrow$

Questions

1. What phenomenon is likely occurring?
2. Give at least three possible interventions.

Answer 22 – Diagnosing Training

The behavior is characteristic of

$$\boxed{\text{overfitting}}.$$

Possible interventions include:

- early stopping;
- weight decay;
- dropout;
- reducing model capacity;
- increasing or augmenting the training data;
- improving the data split or regularization strategy.

Question 23 – Harder: Training and Validation Loss [Level 3]

Suppose the training loss continues decreasing while the validation loss first decreases and then increases.

Questions

1. At which point would you normally select the model?
2. Why should the final epoch not necessarily be selected?
3. Which dataset should be used to estimate final generalization?

Answer 23 – Training and Validation Loss

The model would typically be selected using the validation loss, for example at

$$t^* = \arg\min_t L_{\mathrm{val}}(t).$$

The final epoch may have overfit the training data.
After the model and hyperparameters have been selected, the test set should be used for the final generalization estimate.

Questions: MLPs and Neural Network Training

Topic 6 — Data Splits and Data Leakage

Question 24 – Train / Validation / Test [Level 1]

An engineer trains a neural network on all 10,000 available examples and reports an accuracy of 97

Questions

1. Is this sufficient to establish generalization?
2. What are the respective purposes of training, validation and test sets?

Answer 24 – Train / Validation / Test

Training accuracy alone does not provide a reliable estimate of generalization.

- **Training**: estimate model parameters.
- **Validation**: select hyperparameters and models.
- **Test**: final evaluation of generalization.

The test set should not repeatedly influence model development.

Question 25 – Data Leakage [Level 2]

A student computes

$$\mu = \text{mean of the complete dataset}$$

and

$$\sigma = \text{standard deviation of the complete dataset}$$

and standardizes the data before splitting into training and test sets.

Question

What is the problem? How should the preprocessing be performed?

Answer 25 – Data Leakage

This introduces data leakage because the test set contributes to the preprocessing parameters.
Instead,

$$\mu_{\text{train}}, \qquad \sigma_{\text{train}}$$

must be estimated using the training data only.
Then

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

is applied to validation and test examples.

Question 26 – Harder: Hyperparameter Leakage [Level 3]

An engineer tries 50 different neural-network architectures and learning rates.
For each experiment, the test accuracy is recorded.
The engineer selects the configuration with the highest test accuracy and reports that accuracy as the final result.

Question

Why is this problematic, even though the test examples were never used to compute gradients?

Answer 26 – Hyperparameter Leakage

The test set has influenced model selection.
Although the test examples were not used in gradient computation, the engineer effectively optimized the choice of hyperparameters against the test set.
The test set is therefore no longer an independent final evaluation.
A proper procedure is:

$$\boxed{\text{Training} \rightarrow \text{Validation/model selection} \rightarrow \text{Final test}}$$

Questions: MLPs and Neural Network Training

Topic 7 — Modelling Errors and Target Representation

Question 27 – Wrong Target Representation [Level 2]

A problem has three mutually exclusive classes:

$$y \in \{A, B, C\}.$$

The engineer encodes

$$A = 0, \qquad B = 1, \qquad C = 2$$

and argues:

> *"Class C is twice as large as class A, so the network should learn that C is further away from A."*

Question

Why is this interpretation problematic?

Answer 27 – Wrong Target Representation

The values

$$0,\ 1,\ 2$$

are categorical identifiers.
They do not imply that

$$A < B < C$$

or that the distance between classes has meaning.
For ordinary multiclass classification, the output consists of class logits and the target identifies the correct class.
Numerical target encoding must not introduce structure that does not exist in the problem.

Question 28 – Harder: Ordinal vs Nominal Targets [Level 3]

Consider two problems.

Problem A:

$$y \in \{\text{cat}, \text{dog}, \text{horse}\}.$$

Problem B:

$$y \in \{\text{low}, \text{medium}, \text{high}\}.$$

Question

Can both problems be treated identically as ordinary multiclass classification? What additional structure exists in Problem B?

Answer 28 – Ordinal vs Nominal Targets

Problem A is nominal: the categories have no natural ordering.
Problem B has an ordinal structure:

$$\text{low} < \text{medium} < \text{high}.$$

A standard multiclass formulation is still possible, but it does not explicitly exploit the ordering.
For ordinal prediction, specialized formulations can exploit this additional structure.

Key modelling principle

The representation should reflect meaningful structure in the prediction problem, but should not invent structure that does not exist.

Questions: MLPs and Neural Network Training

Topic 8 — Advanced Optimization and Numerical Issues

Question 29 – Harder: Gradient Sign [Level 2]

Suppose

$$\frac{\partial L}{\partial \theta} < 0.$$

Questions

1. In which direction does increasing θ initially change the loss?
2. In which direction will gradient descent move θ?

Answer 29 – Gradient Sign

If

$$\frac{\partial L}{\partial \theta} < 0,$$

then locally increasing θ decreases the loss.
Gradient descent performs

$$\theta_{\text{new}} = \theta - \eta \frac{\partial L}{\partial \theta}.$$

Since the derivative is negative,

$$\theta_{\text{new}} > \theta.$$

Thus gradient descent increases θ.

Question 30 – Harder: Vanishing Gradients [Level 3]

Consider a deep network using sigmoid activations.
The derivative of the sigmoid satisfies

$$0 < \sigma'(z) \leq \frac{1}{4}.$$

Question

Why can gradients become extremely small when propagated through many layers?

Answer 30 – Vanishing Gradients

Backpropagation repeatedly applies the chain rule.
Gradients therefore contain products of derivatives such as

$$\prod_{l=1}^{L} \sigma'(z_l) W_l.$$

If many factors have magnitude smaller than one, the product can become exponentially small as the depth increases.
This is the **vanishing-gradient problem**.
It can make early layers learn very slowly.
ReLU-like activations, residual connections and appropriate initialization are among the techniques used to mitigate it.

Question 31 – Harder: Exploding Gradients [Level 3]

The opposite phenomenon can also occur.

Question

What can cause gradients to become extremely large in a deep network? Give two possible mitigation strategies.

Answer 31 – Exploding Gradients

Repeated multiplication by matrices with large effective gain can produce gradients whose norm grows rapidly:

$$\left\| \prod_l W_l \right\| \gg 1.$$

Possible causes include inappropriate initialization or unstable optimization dynamics.
Possible mitigation strategies include:

- gradient clipping;
- appropriate parameter initialization;
- normalization techniques;
- reducing the learning rate;
- architectural mechanisms such as residual connections.

Question 32 – Harder: Numerical Stability [Level 3]

For multiclass classification, softmax is

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Suppose

$$z = (1000, 999, 998).$$

Questions

1. Why can computing e^{z_i} directly be numerically problematic?
2. How can softmax be computed more stably?

Answer 32 – Numerical Stability

Directly computing e^{1000} can overflow numerically.
Because softmax is invariant to adding the same constant to all logits,

$$\mathrm{softmax}(z) = \mathrm{softmax}(z - c).$$

Choosing

$$c = \max_i z_i$$

gives

$$p_i = \frac{e^{z_i - \max_j z_j}}{\sum_j e^{z_j - \max_k z_k}}.$$

Now the largest exponent is

$$e^0 = 1$$

This is one reason numerically stable implementations combine operations such as log-softmax and cross-entropy internally.

Questions: MLPs and Neural Network Training

Topic 9 — Advanced Model Diagnosis

Question 33 – Harder: Class Imbalance [Level 3]

A binary classifier is evaluated on a dataset containing

- 99% negative examples
- 1% positive examples

A trivial model that always predicts "negative" obtains 99% accuracy.

Questions

1. Why is accuracy misleading in this situation?
2. Name two metrics that provide additional information.
3. Give one possible training intervention.

Answer 33 – Class Imbalance

Accuracy does not distinguish between the two classes and can be dominated by the majority class.
Useful additional metrics include:

$$\text{precision}, \qquad \text{recall}, \qquad F_1, \qquad \text{ROC-AUC}..$$

Depending on the problem, training can also use class-weighted losses, resampling, or other imbalance-aware strategies.
The appropriate metric should reflect the application objective.

Question 34 – Harder: Threshold vs Logit [Level 3]

A binary classifier produces a logit

$$z = 0.8.$$

Questions

1. What is the corresponding probability?
2. Under a threshold of 0.5, what class is predicted?
3. What logit corresponds exactly to a probability of 0.5?
4. Is the threshold 0.5 necessarily optimal for every application?

Answer 34 – Threshold vs Logit

The probability is

$$p = \sigma(0.8) = \frac{1}{1 + e^{-0.8}} \approx 0.69.$$

Thus a threshold of 0.5 produces the positive class.
Since

$$\sigma(0) = 0.5,$$

the corresponding logit is

$$\boxed{z = 0}.$$

The threshold does not necessarily have to be 0.5. It can be chosen according to the relative costs of false positives and false negatives or other application requirements.

Question 35 – Harder: Model Capacity [Level 3]

Two models are trained on the same dataset.

	Training accuracy	Validation accuracy
A	72%	70%
B	99%	71%

Questions

1. What does the difference between training and validation performance suggest for Model B?
2. Does Model A necessarily have "too little capacity"?
3. Give two ways of investigating the situation further.

Answer 35 – Model Capacity

Model B shows a large generalization gap:

$$99\% - 71\% = 28\%.$$

This is consistent with substantial overfitting.
Model A may be underfitting, but this cannot be concluded solely from the two accuracy values.
Further investigation could include:

- examining training and validation loss curves;
- increasing or decreasing model capacity;
- testing regularization;
- checking data quality and label noise;
- examining whether the train/validation distributions differ.

Questions: MLPs and Neural Network Training

Topic 10 — Integrated Reasoning

Question 36 – Integrated Modelling Problem [Level 3]

You must build a neural network to predict whether each of four independent faults is present in a machine.
The dataset contains 20 input features.

Question

Propose:

1. the output dimension;
2. the output representation;
3. an appropriate loss;
4. the interpretation of the output;
5. an appropriate prediction rule;
6. one appropriate evaluation metric.

Answer 36 – Integrated Modelling Problem

This is a multilabel classification problem.
A standard formulation is

$$\text{Linear}(h, 4)$$

producing four logits:

$$\mathbf{z} \in \mathbb{R}^4.$$

Use

$$\boxed{\texttt{BCEWithLogitsLoss}}.$$

Each logit corresponds to an independent binary prediction.
Probabilities are

$$p_i = \sigma(z_i).$$

A threshold can then be applied independently:

Question 37 – Integrated Debugging [Level 3]

```
model = nn.Sequential(
    nn.Linear(20, 64),
    nn.ReLU(),
    nn.Linear(64, 4),
    nn.Softmax(dim=1)
)
criterion = nn.CrossEntropyLoss()

# preprocessing:
X = standardize(all_data)
...
# training:
for x, y in dataloader:
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
optimizer.step()
```

Question

Identify as many conceptual or implementation problems as you can. Explain why each one is problematic.

Answer 37 – Integrated Debugging

Several issues should be identified.

1. **Softmax before CrossEntropyLoss:**
   “‘ The model should normally output logits directly.
2. **Missing `zero_grad()`:**
   Gradients accumulate by default in PyTorch. If accumulation is not intentional, call `optimizer.zero_grad()`.
3. **Potential preprocessing leakage:**
   If `all_data` contains validation or test examples, the standardization statistics leak information from those sets.
4. **Data splitting should precede fitting preprocessing:**
   Compute preprocessing statistics on the training set only and apply the resulting transformation to validation and test data.
5. **Interpretation of the output:**
   The four logits correspond to four mutually exclusive classes only if this is a multiclass problem. If the four outputs instead represent independent faults, BCEWithLogitsLoss would be appropriate instead.

Question 38 – Final Synthesis [Level 3]

Complete the following chain:

$$\boxed{\text{Problem} \rightarrow ? \rightarrow ? \rightarrow ? \rightarrow ? \rightarrow \text{Evaluation}}$$

Question

Explain the main modelling decisions that connect the original problem to the final evaluation of a neural network.

Answer 38 – Final Synthesis

A useful conceptual chain is

$$\text{Problem} \to \text{Target representation} \to \text{Model/output representation} \to \text{Loss} \to \text{Optimization} \to \text{Evaluation}$$

The choices must be mutually consistent.
For example:
binary target $\to$ one logit $\to$ BCEWithLogitsLoss
$\to$ gradient-based optimization $\to$ appropriate test metric.

A large part of successful neural-network modelling is therefore not merely choosing an architecture, but ensuring that the **problem formulation, target representation, output layer, loss, optimization procedure and evaluation protocol are coherent**.